ME 295 Lab Report 1
AI Agents for Optimization of Thin-Shell Composites

Tara Venkatadri
October 16th, 2026

Project Topic

I will be working with Federico Benazzo on the class project. We are both aerospace PhD students in Professor Sergio Pellegrino's lab, and our research relates to thin-shell carbon fiber reinforced polymer (CFRP) booms used in deployable spacecraft structures. During their lifetimes, these booms are subjected to many different types of loading. In addition to typical load cases (such as beam bending, axial extension and torsion, and axial compression inducing buckling), they undergo specific types of loads related to their folding and deployment (e.g. coiling around a central hub and folding in the middle of the boom).

Due to the complex nature of these loads, it is often difficult to determine analytically the best design that minimizes the boom's mass and packaged volume (to reduce launch costs) without experiencing material failure. Examples of design variables include the raw materials being used, the choice of lamina orientations and layup, and the geometric parameters defining the cross-sectional shape of the boom. We propose to develop a team of AI agents to optimize the design of these composite booms under various load cases, using the finite element Python plug-in OpenSeesPy as part of the agents' workflow. Although OpenSeesPy is typically used for civil engineering applications (e.g. testing earthquake loading), it has capabilities for shell elements and orthotropic materials that make it applicable to our problem as well.

Relevant Literature and Novel Aspects of Project

There is extensive literature on the use of thin-shell composite materials for deployable space structures, much of which comes from the Pellegrino lab. Specifically focusing on the optimization of these structures, a relevant study was conducted by Liu et al. (2023). The authors conducted a multi-objective optimization of the geometric parameters of lenticular deployable composite booms using a genetic algorithm. They sought to minimize the coiling radius and moment, minimize the boom's mass per unit length, and maximize the moments of area of the cross-section to provide flexural stiffness and increase the natural frequency, subject to constraints on material strengths and other properties. From the optimization, they identified designs on the Pareto front that were optimal with respect to some of their objectives, ultimately choosing a design that was better than their initial test sample with respect to all objectives.

Other aspects of the literature focus on the inner workings of OpenSeesPy. The foundational paper introducing OpenSeesPy was written by Zhu et al. (2018). It describes the newly developed Python link to the open-source finite element framework OpenSees (Open System for Earthquake Engineering Simulation) and provides a worked example of the way the software is used. As with other finite element solvers, the nodes and elements are first defined, followed by material properties. Loads and constraints are specified, and a static or dynamic analysis is performed. There does not appear to be any literature on the use of OpenSeesPy for thin-shell carbon fiber composites, but there are several features in the OpenSeesPy documentation that appear useful for modeling these types of problems. Geometrically nonlinear shell elements capable of large deformation are modeled using the ShellNLDKGT and ShellNLDKGQ features for triangle and quadrilateral elements, respectively. The material

properties of each layer of the composite are specified using the ElasticOrthotropic function, and the layers are combined to produce the full layup using the LayeredShell property.

A third field of relevant literature discusses the use of large language models (LLMs) to create agents that run structural analysis in OpenSeesPy. Liu et al. (2026) first attempted to solve beam bending problems using LLMs directly and found that as the problems move away from canonical problems toward combined load cases and loads applied in unusual locations, the accuracy of LLMs decreases. They found that the true potential of LLMs could be harnessed by reframing structural analysis problems as code generation tasks to run existing finite element solvers, such as OpenSeesPy. Their framework involves LLM-powered agents that take user input in natural language, parse it to define the geometry, boundary conditions, and loads on the structure, write the OpenSeesPy code, then run the finite element analysis and use the OpsVis platform in Python to visualize the structure's behavior and help users to interpret the results. The results of these agents were far more reliable and robust than having the LLM perform the structural analysis itself as a text generation task. Another example of an LLM agent team for OpenSeesPy is provided by Zhang et al. (2026), who created a team of highly specialized agents to write, run, and validate OpenSeesPy programs to design and analyze steel structures. A particularly useful feature of this paper is a series of sample interactions between the user and LLM, followed by the resulting code written by the LLM. It may be useful for us to mirror this structure as we are beginning to set up our own team of agents.

The above studies have separately investigated the optimization of thin-shell composites, the use of OpenSeesPy to solve structural mechanics problems (including with thin orthotropic shells), and the ability of LLMs to write and run finite element analysis in OpenSeesPy. However, no study to date has combined these concepts to use LLM-powered agents to optimize thin-shell composites in OpenSeesPy. This will be the gap in literature that our project addresses.

Proposed AI Agent Workflow

The user will input several key pieces of information to the program in natural language. These will include the available materials and their stiffness and strength parameters, the class of cross-section and loading case under consideration, the desired property to optimize, and any constraints on the optimization aside from material strength. The final output of the system will be an optimized design (subject to the constraints) for a CFRP boom of the desired cross-section type, with materials, layup, and geometric parameters specified.

The tasks between the initial input and final output will be broken down into pieces, with a specialized Claude-powered AI agent addressing each one. First, an agent will need to convert the user's natural-language optimization scheme into a mathematical representation with an objective function and constraints. Next, a starting point for the optimization must be chosen by a second agent, with an initial guess of the material, layup, and geometry. A team of agents will, for a given test sample, convert the material properties, geometry, loading, and boundary conditions into OpenSeesPy code and generate the finite element mesh. Another agent will run the Python code to complete the finite element analysis and visualize the key results using the OpsVis library. A final agent will analyze the output for this sample and compare it to the objective function and constraints, then suggest adjustments to the choices of materials, layup, and geometric parameters to work toward an optimal solution. These agents will run in an iterative loop until an optimal solution has been identified, at which point the final answer will be provided to the user.

During the development of these agents, we will not treat them as a "black box" – instead, we will verify at each step that they are performing the desired actions. For the optimization definition agent, we will ask it to output its objective function and constraints in equation form, so that we can ensure that they match what we expect for several test cases of natural language input. For the agent that defines the OpenSeesPy material properties, we will create secondary agents that read the Python code and convert it to an ABD matrix that we can check against established values from papers (e.g. Palmeri and Pellegrino, 2026). For the agent that defines the geometry, mesh, and loading, we will similarly create an agent that translates the Python code to a schematic figure with dimensions and loads labeled, to verify that they are as intended before running the finite element code. To ensure that the OpenSeesPy finite element analysis runs as intended, we will check the results from OpsVis against Abaqus models we will develop by hand. Finally, for the optimization iteration agent, we will ask it to output the properties of the sample it tested, the value of the objective, and its reasoning for why it will change specific parameters in the next iteration. It will be difficult to check that a truly optimal solution has been found, but by reading the trace to look through the iterations, we can ensure that it is taking a reasonable approach to move through the design space.

After developing the team of agents, we will test our system on a series of load cases, cross-sections, and objective functions. First, we will start with simple tests to ensure that the pipeline works as intended. These will include looking at flat CFRP strips under axial tension, compression, and twist, as well as bending loads under simply-supported and cantilevered boundary conditions. Next, we will branch out to other cross-sectional geometries more commonly used for deployable structures, such as curved tape springs, triangular rollable and collapsible booms, and lenticular booms (Murphey et al., 2026). Depending on the capabilities of OpenSeesPy, we may also attempt to study load cases specific to deployable structures, such as coiling and folding at the midpoint of the boom, which will require the introduction of contact formulations. We will start with an objective function to minimize mass, but may later look at minimizing packaged volume for launch or the ratio of packaged to deployed volume, or potentially maximizing material failure criteria and buckling strength to produce structures that can withstand higher loads.

Progress to Date

We have been focused until now on setting up the technology necessary to begin the project. We have set up our Amazon WorkSpaces with Visual Studio Code and Claude Code. We have tested that Claude Code is able to write a simple "hello world" Python script, and that we can open the code in Visual Studio Code. We cannot run the code because Python is not installed on the Amazon WorkSpace, but we have spoken with Alice and she will look into it.

Federico also created a GitHub repository and shared it with me, and we confirmed that we can both add files and edit files created by the other person. We have synchronized our Visual Studio desktop apps with the GitHub folder so that we can easily make changes to the code. We have set up a file structure in GitHub with placeholder folders for documents that we anticipate creating over the course of the project, including a running log as a .txt file that currently documents our initial setup actions and will later document our progress on the project. The GitHub repository has been copied to the class workspace and is available at https://github.com/citgitfr/BenazzoVenkatadri_ThinShellSims/tree/main.

In the coming weeks, we look forward to making technical progress on the project by beginning to set up and test our team of AI agents.

References

Liu, J., Geng, Z., Cao, R., Cheng, L., Bocchini, P., & Cheng, M. (2026). A large language model-empowered agent for reliable and robust structural analysis. *Structure and Infrastructure Engineering*, 1-16.

Liu, T. W., Bai, J. B., & Fantuzzi, N. (2023). Folding behavior of the thin-walled lenticular deployable composite boom: Analytical analysis and many-objective optimization. *Mechanics of Advanced Materials and Structures*, *30*(11), 2221-2239.

Murphey, T. W., Hunt, D., Browning, D., & Folkers, M. W. (2026). Performance Comparison of Flattenable and Rollable Booms. In *AIAA SCITECH 2026 Forum* (p. 0802).

Palmeri, F., & Pellegrino, S. (2026). Performance-driven design of deployment mechanism for collapsible tube masts. In *AIAA SCITECH 2026 Forum* (p. 1041).

The OpenSeesPy Library. The OpenSeesPy Library - OpenSeesPy 3.5.1.3 documentation. (n.d.). https://openseespydoc.readthedocs.io/en/latest/

Zhang, Z., Chen, K., Cevik, C. B., & Moen, C. D. (2026). LLM agent framework for structural steel design. In *Proceedings of the Annual Stability Conference, Structural Stability Research Council*.

Zhu, M., McKenna, F., & Scott, M. H. (2018). OpenSeesPy: Python library for the OpenSees finite element framework. *SoftwareX*, *7*, 6-11.